

Maestría en
Software

Patrones de Diseño de Software

Autor(es): Mauricio Ortiz Ochoa
mortizo@ups.edu.ec

Tema:

Unidad 03. Patrones de diseño de comportamiento

Tema:

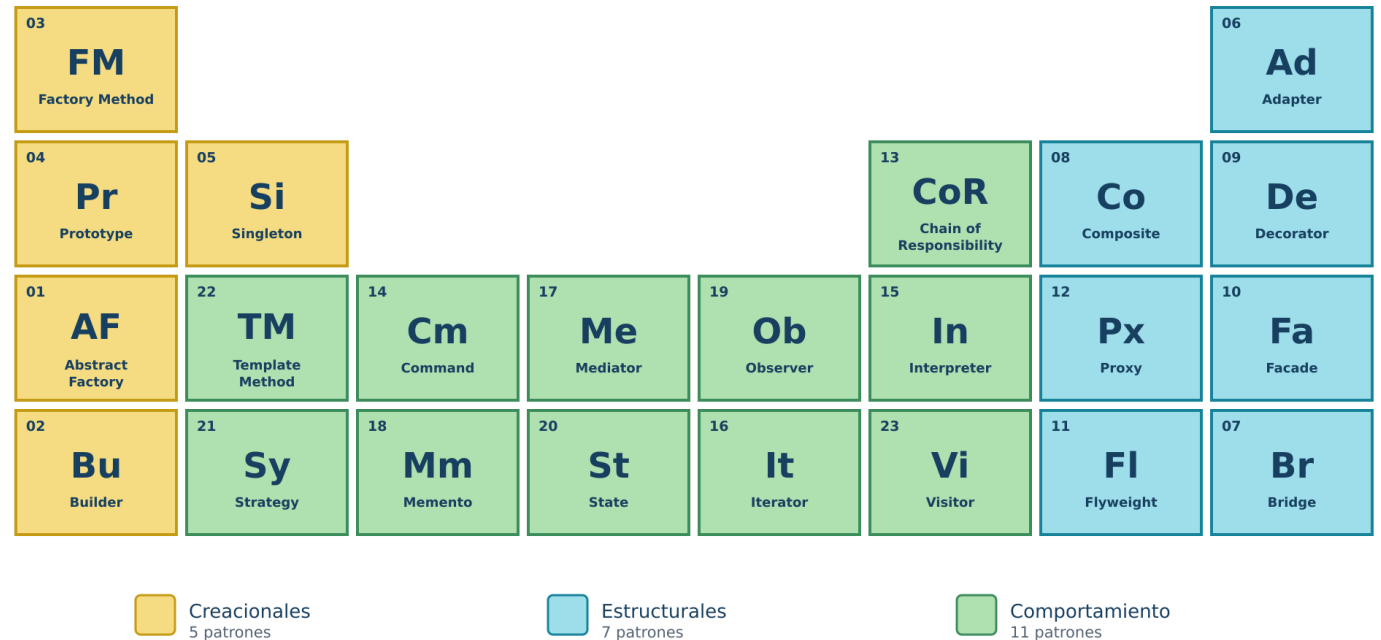
Patrones de comportamiento

- *Patrones de comportamiento*
- Strategy
- Template Method

Introducción a los patrones de comportamiento

Clasificación de los patrones GoF por propósito

- **Patrones creacionales (5):** Abstraen el proceso de creación de objetos y reducen la dependencia respecto de clases concretas.
- **Patrones estructurales (7):** Definen cómo combinar clases y objetos para construir estructuras mayores, flexibles y mantenibles.
- **Patrones de comportamiento (11):** Organizan cómo los objetos distribuyen responsabilidades, colaboran y varían su comportamiento para realizar una tarea.



Catálogo GoF: 23 patrones de diseño

Patrones de comportamiento

Introducción

- Los patrones de comportamiento se ocupan de algoritmos, responsabilidades e interacciones entre objetos.
- Permiten distribuir el comportamiento entre diferentes participantes evitando concentrar demasiadas decisiones en una sola clase.
- Describen cómo colaboran los objetos para realizar una tarea y cómo pueden variar esas colaboraciones.
- Los patrones de comportamiento de clase utilizan principalmente la herencia para distribuir comportamiento.
- Los patrones de comportamiento de objetos utilizan principalmente composición y delegación para organizar responsabilidades dinámicamente.

Patrones de comportamiento

Catálogo

- State: Permite que un objeto modifique su comportamiento cuando cambia su estado interno.
- Template Method: Define el esqueleto de un algoritmo y permite que las subclases redefinan determinados pasos sin cambiar su estructura general.
- Strategy: Define una familia de algoritmos o comportamientos, los encapsula y permite intercambiarlos de forma independiente del contexto que los utiliza.
- Observer: Define una relación uno-a-muchos para notificar automáticamente a los objetos interesados cuando cambia el estado de un sujeto.
- Mediator: Centraliza y coordina las interacciones entre un conjunto de objetos para reducir sus dependencias directas.
- Chain of Responsibility: Permite pasar una solicitud a través de una cadena de posibles receptores hasta que alguno pueda atenderla.
- Command: Encapsula una solicitud como un objeto, permitiendo parametrizar, registrar, deshacer o diferir operaciones.
- Interpreter: Define una representación para la gramática de un lenguaje y un mecanismo para interpretar expresiones de ese lenguaje.
- Iterator: Permite recorrer secuencialmente los elementos de una colección sin exponer su representación interna.
- Memento: Captura y permite restaurar el estado interno de un objeto sin violar su encapsulación.
- Visitor: Permite agregar operaciones sobre una estructura de objetos sin modificar las clases de los elementos sobre los que opera.

Tema:

Strategy

- Patrones de comportamiento
- *Strategy*
- Template Method

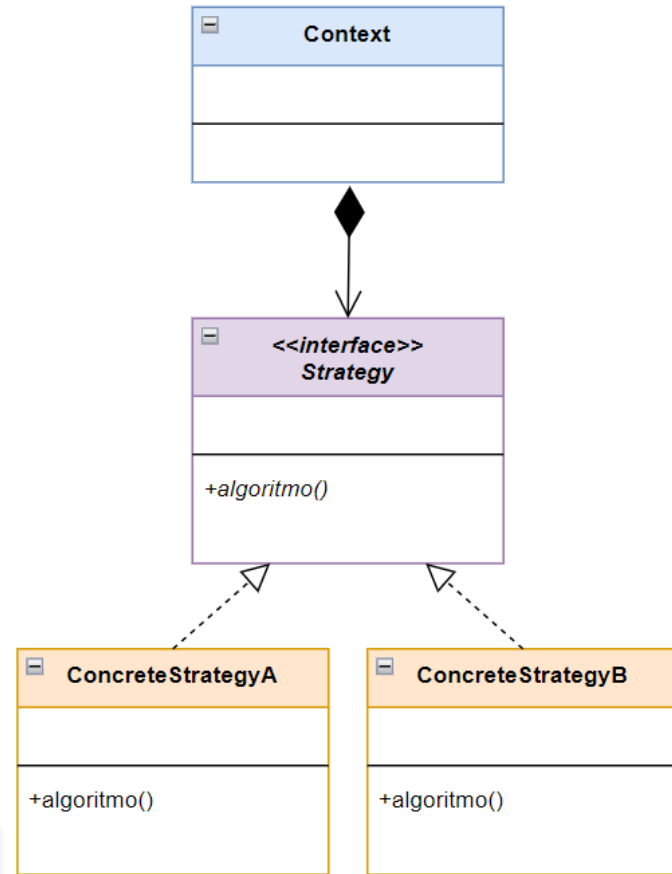
Strategy

Strategy

- Propósito
 - Define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables. Permite que un algoritmo varíe independientemente de los clientes que lo utilizan.
- Problema / Motivación
 - Supongamos que en una Universidad existen diferentes políticas para calcular el descuento por pronto pago según el tipo de programa académico.
 Grado: descuento del 10 % sobre la cuota, independientemente del número de cuotas pagadas.
 Posgrado: descuento del 15 % cuando se pagan al menos 3 cuotas; con 1 o 2 cuotas no se aplica descuento.
 Tecnologías: descuento del 10 % sobre la cuota, pero únicamente para asignaturas del primer ciclo.
 - A medida que aparecen nuevos programas o cambian las políticas de descuento, una única operación de cálculo puede empezar a acumular múltiples condicionales y requerir modificaciones continuas.

Strategy

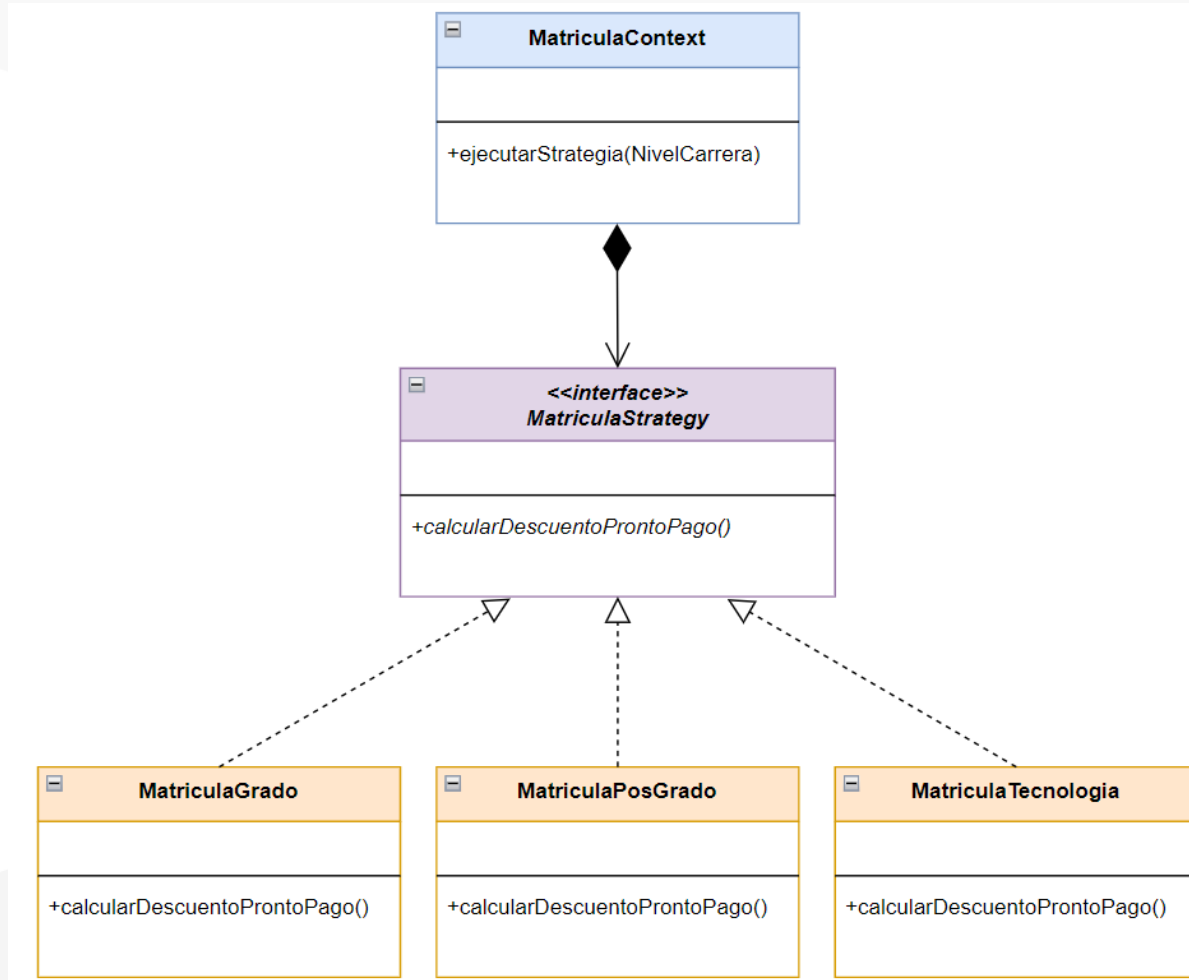
Estructura



- Participantes
 - **Context**
Se configura con un objeto **ConcreteStrategy**.
Mantiene una referencia a un objeto **Strategy**.
Puede proporcionar a la estrategia los datos necesarios para ejecutar el algoritmo.
 - **Strategy**
Declara una interfaz común para todos los algoritmos disponibles.
Context utiliza esta interfaz para delegar la ejecución del comportamiento seleccionado.
 - **ConcreteStrategy**
Implementa una variante específica del algoritmo definido por **Strategy**.
- Colaboraciones
 - Context y Strategy colaboran para ejecutar el algoritmo seleccionado.
 - El cliente normalmente crea una **ConcreteStrategy** y la proporciona al Context.
 - Posteriormente, el cliente interactúa con el Context, mientras este delega el comportamiento en la estrategia configurada.

Strategy

Solución



<https://github.com/mortizo/pds-unidad03-ooms/tree/master/pds-unidad03-ooms/src/main/java/p69/strategy>

Strategy

Strategy

- Aplicabilidad
 - Cuando existan varias variantes de un algoritmo o comportamiento que deban poder intercambiarse.
 - Varias clases relacionadas difieran principalmente en su comportamiento.
 - Una clase concentre múltiples condicionales para seleccionar diferentes comportamientos.
 - Se necesite aislar la lógica específica de un algoritmo del contexto que lo utiliza.
 - Se quiera incorporar nuevas variantes sin modificar continuamente el contexto.
- Evite utilizarlo cuando:
 - Solo exista una variante estable del comportamiento.
 - Las diferencias entre alternativas sean simples y poco probables de cambiar.
 - Un condicional pequeño y claro resuelva suficientemente el problema.
 - La abstracción adicional introduzca más complejidad que beneficio.

Strategy

Actividad en clase: ¿Strategy?

Caso 1 — Sistema de recomendación de películas

Una plataforma debe recomendar películas utilizando diferentes criterios: género preferido, popularidad o similitud entre usuarios. Los criterios pueden cambiar según el perfil o configuración del usuario.

¿Cómo diseñarían el sistema para incorporar nuevos criterios de recomendación sin modificar continuamente el componente que solicita las recomendaciones?

Caso 2 — Gestión de inventario en una tienda en línea

Una tienda necesita aplicar diferentes políticas para determinar qué productos deben salir primero del inventario: FIFO, LIFO o una política basada en demanda. La política utilizada puede variar según el tipo de producto.

¿Cómo diseñarían el sistema para cambiar la política utilizada sin modificar la lógica general de gestión del inventario?

Caso 3 — Procesamiento de pagos en una tienda en línea

Una tienda permite pagar mediante tarjeta de crédito, transferencia bancaria o pago en efectivo. Cada modalidad requiere un procedimiento diferente para procesar el pago y generar el comprobante.

¿Cómo diseñarían el procesamiento para incorporar nuevas modalidades de pago sin modificar el flujo principal de compra?

Caso 4 — Vehículos en una carrera

Los vehículos pueden utilizar diferentes políticas de aceleración: turbo, normal o económica.

Cada uno calcula de forma diferente la velocidad y la aceleración, y un vehículo puede cambiar de política durante la carrera.

¿Cómo diseñarían el sistema para intercambiar estas formas de aceleración sin modificar la clase que representa al vehículo?

Tema:

Template Method

- Patrones de comportamiento
- Strategy
- *Template Method*

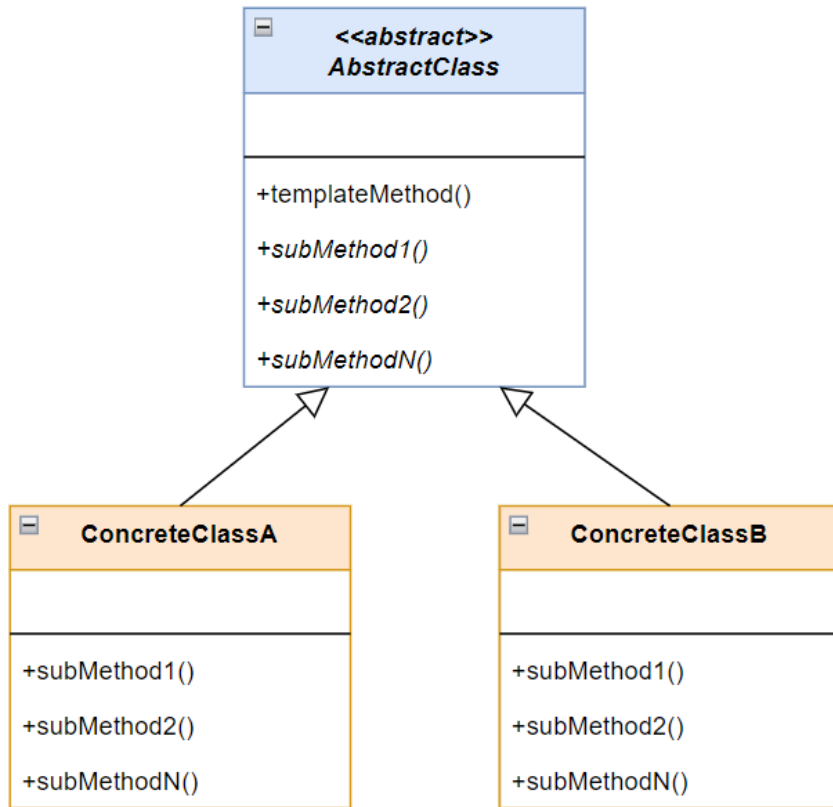
Template Method

Template Method

- Propósito
 - Define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos.
 - Permite que las subclases redefinan determinados pasos sin modificar la estructura general del algoritmo.
- Problema / Motivación
 - Supongamos que una empresa calcula el sueldo de sus empleados siguiendo siempre la misma secuencia:
 1. Calcular el salario básico.
 2. Calcular valores adicionales.
 3. Calcular descuentos.
 4. Obtener el sueldo final.
 - Para los empleados fijos, el salario básico corresponde a las horas trabajadas $\times$ \$10; reciben además un bono de \$5 y tienen un descuento del 9 % del salario básico por seguridad social.
 - Para los empleados temporales, el salario básico corresponde a las horas trabajadas $\times$ \$12; no reciben bono ni tienen descuento por seguridad social.
 - Necesitamos mantener una única secuencia para calcular el sueldo, permitiendo que determinados pasos varíen según el tipo de empleado.

Template Method

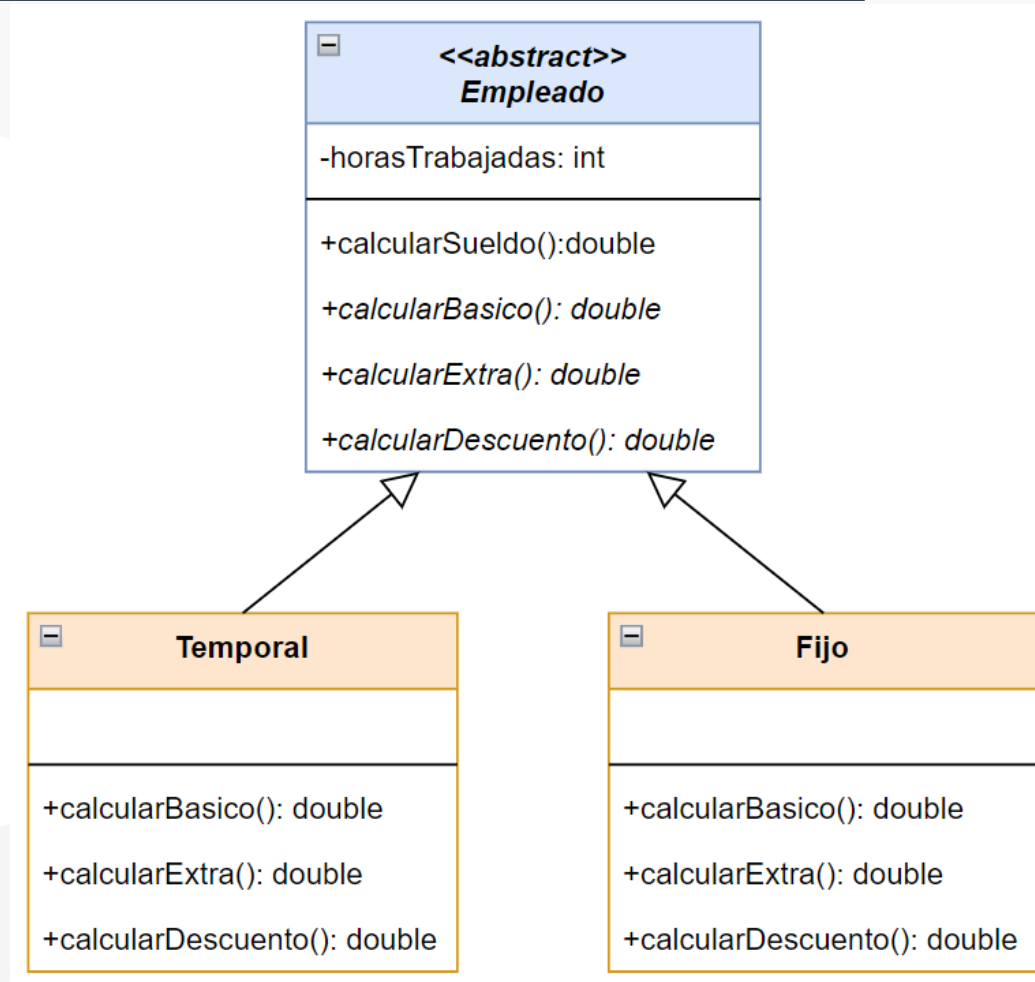
Estructura



- Participantes
 - **AbstractClass:**
Define e implementa el `templateMethod()` con el esqueleto y orden general del algoritmo.
Declara las operaciones primitivas que pueden ser implementadas o redefinidas por las subclases.
 - **ConcreteClass:**
Implementa las operaciones primitivas necesarias para completar los pasos variables del algoritmo.
No modifica la estructura general definida por el `templateMethod()`.
- Colaboraciones
 - **AbstractClass** controla la secuencia general del algoritmo mediante `templateMethod()`.
 - `templateMethod()` invoca las operaciones primitivas en el orden definido.
 - Las **ConcreteClass** proporcionan la implementación específica de los pasos variables.

Template Method

Solución



<https://github.com/mortizo/pds-unidad03-ooms/tree/master/pds-unidad03-ooms/src/main/java/p69/template>

Template Method

Template Method

- Aplicabilidad
 - Exista un proceso general estable cuyo orden no deba cambiar.
 - Algunas etapas del algoritmo deban variar entre diferentes subclases.
 - Exista comportamiento común entre varias clases que pueda centralizarse en una clase base.
 - Se necesiten puntos de extensión controlados sin permitir que las subclases alteren libremente el flujo completo.
- Evite utilizarlo cuando
 - Cada variante requiera un algoritmo completamente diferente.
 - Se necesite cambiar el comportamiento dinámicamente en tiempo de ejecución.
 - La jerarquía de herencia introduzca más rigidez que beneficio.
 - El proceso tenga pocos pasos y la abstracción adicional no aporte valor.

Template Method

Actividad en clase: ¿Template Method?

Caso 1 — Registro de estudiantes

Una institución registra estudiantes de tiempo completo y medio tiempo. Todos atraviesan el mismo proceso general de registro, pero cambian algunas validaciones, como carga académica mínima y requisitos de asistencia.

¿Cómo diseñarían el proceso para mantener una secuencia común y permitir que determinados pasos cambien según el tipo de estudiante?

Caso 2 — Gestión de productos

Una tienda procesa productos perecederos y no perecederos. Todos deben ser recibidos, registrados, almacenados y preparados para despacho, pero algunos pasos de almacenamiento y control cambian según el tipo de producto.

¿Cómo diseñarían el proceso para conservar el flujo general y especializar únicamente los pasos que realmente varían?

Caso 3 — Generación de reportes

Un sistema genera reportes en PDF, Excel y HTML. Todos siguen la misma secuencia: obtener datos, procesarlos, darles formato y generar el archivo, pero algunos pasos dependen del formato seleccionado.

¿Cómo diseñarían el proceso para evitar repetir la secuencia completa en cada tipo de reporte?

Caso 4 — Importación de archivos

Una aplicación importa información desde archivos CSV, JSON y XML. El proceso siempre valida el archivo, lee los datos, los transforma y los almacena, aunque la forma de leer y transformar cambia según el formato.

¿Cómo diseñarían el proceso para mantener una estructura común y permitir variaciones controladas en determinados pasos?

Unidad 03.01

Retroalimentación y cierre

Qué vimos hoy

- Los patrones de comportamiento organizan responsabilidades, algoritmos e interacciones entre objetos.
- Strategy encapsula comportamientos intercambiables y permite seleccionarlos mediante composición.
- Template Method mantiene estable el esqueleto de un algoritmo y permite variar determinados pasos mediante herencia.

Antes de finalizar:

¿Qué comportamiento necesita poder intercambiarse? → Strategy

¿Qué parte del algoritmo permanece estable y qué pasos pueden variar? → Template Method

¿Quién controla la variación: un objeto colaborador o una subclase?

¿Realmente necesitamos un patrón o una solución más simple sería suficiente?

Idea final: Strategy cambia el comportamiento mediante composición; Template Method mantiene un proceso común y permite variar pasos mediante herencia.

Anexos

Bibliografía

Gamma, E., Helm, R., Johnson, R. y Vlissides, J.; Design Patterns: Elements of Reusable Object-Oriented Software; 1.ª edición, 1994.

Sarcar, V.; Java Design Patterns: A Hands-On Experience with Real-World Examples; 3.ª edición, 2022.

Musch, O.; Design Patterns with Java: An Introduction; 1.ª edición, 2023.

Fowler, M.; UML Distilled: A Brief Guide to the Standard Object Modeling Language; 3.ª edición, 2004.

Shvets, A.; Sumérgete en los patrones de diseño; 1.ª edición, 2019.

IEEE Computer Society; Guide to the Software Engineering Body of Knowledge — SWEBOK Guide V4.0a; versión 4.0a, 2025.

Asaad, J. y Avksentieva, E.; Assessing the Impact of GoF Design Patterns on Software Engineering Practices; 2025.

Gracias por su atención

posgrados.ups.edu.ec